

SIMATS ENGINEERING
CO1 ASSESSMENT TOOL 2

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems.

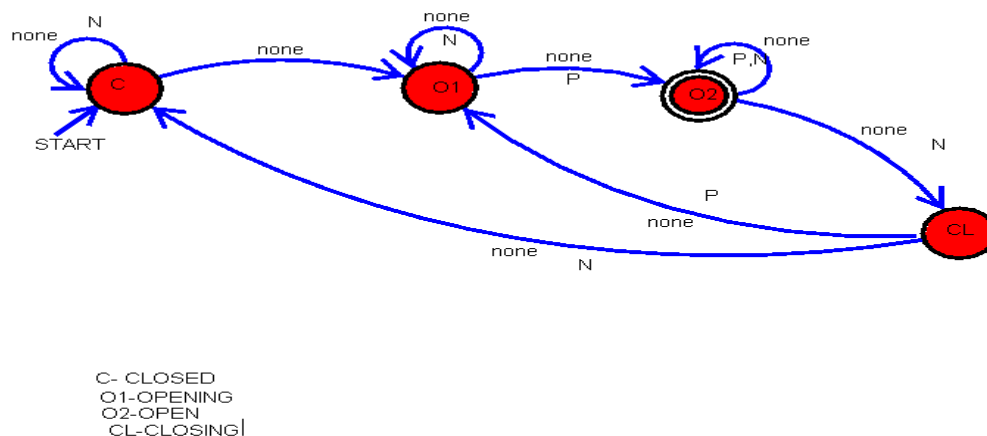
SIMULATION

Code of Conduct:

I **GIRIJA B 192311044** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Girija B

1. **Design a Non-Deterministic Finite Automaton (NBA) for an Automatic Door System based on sensor inputs.** An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non-determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.



Problem Understanding

An Automatic Door System has four states:

- **Closed (C)** – Door is closed (Initial State)
- **Opening (O₁)** – Door is opening
- **Open (O₂)** – Door is completely open
- **Closing (CL)** – Door is closing

Input Symbols

$$\Sigma = \{P, N\}$$

Where,

- **P** = Person Detected
- **N** = No Person Detected

Initially, the door is **Closed**.

When a person is detected (**P**), the door starts opening.

Once opened, it reaches the **Open** state.

If no person is detected (**N**), the system starts closing.

After closing, it returns to **Closed**.

Since this is an **NFA**, when the door is **Open** and another **P** is detected, the system may

- remain Open **or**
- continue operating (non-deterministic behavior).

NFA Design

States

$$Q = \{C, O_1, O_2, CL\}$$

Where

- C = Closed
- O₁ = Opening
- O₂ = Open
- CL = Closing

Alphabet

$$\Sigma = \{P, N\}$$

Start State

$$q_0 = C$$

Final State

Door completely open.

$$F = \{O_2\}$$

Transition Table

Current State	P	N
→C (Closed)	{O ₁ }	{C}
O ₁ (Opening)	{O ₂ }	{O ₁ }
*O ₂ (Open)	{O ₂ }	{O ₂ , CL}
CL (Closing)	{O ₁ }	{C}

Formal Definition

The NFA is

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

States

$$Q = \{C, O_1, O_2, CL\}$$

Alphabet

$$\Sigma = \{P, N\}$$

Transition Function

δ	P	N
C	{O ₁ }	{C}
O ₁	{O ₂ }	{O ₁ }
O ₂	{O ₂ }	{O ₂ , CL}

δ	P	N
CL	$\{O_1\}$	$\{C\}$

Initial State

$$q_0 = C$$

Final State

$$F = \{O_2\}$$

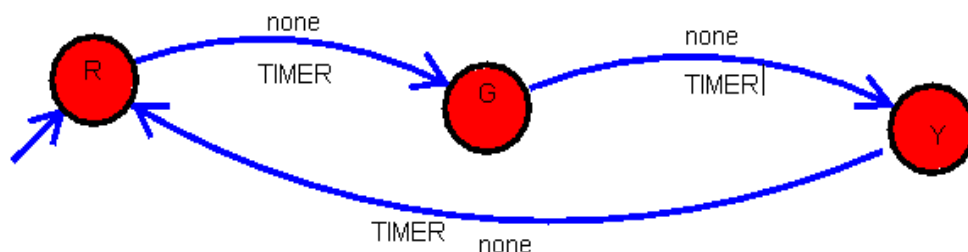
Explanation

- Initially, the door is **Closed**.
- When a person (**P**) is detected, it moves to **Opening**.
- Another **P** moves it to **Open**.
- If no person (**N**) is detected while the door is open, it may either:
 - remain **Open**, or
 - start **Closing**.
- After **Closing**, if no person is detected, the door returns to **Closed**.
- If a person is detected while closing, the door immediately starts **Opening** again.

Conclusion

The designed NFA accurately models the Automatic Door System with four states: **Closed**, **Opening**, **Open**, and **Closing**. The transition from **Open** on input **N** demonstrates **non-determinism**, as the automaton may either remain open or begin closing. This NFA satisfies the required behavior and correctly represents the automatic door system.

- Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.



Conceptual Understanding

A **Finite Automaton (FA)** is used to model systems that move between a finite number of states based on input events. In a traffic light system, the automaton continuously cycles through the three traffic signals in the following order:

Red → Green → Yellow → Red

Each state remains active for a fixed duration before transitioning to the next state.

Finite Automaton Design

States

- **R** – Red Light
- **G** – Green Light
- **Y** – Yellow Light

Alphabet (Input Symbols)

$$\Sigma = \{T\}$$

where

- **T** = Timer Expired

Formal Definition

The automaton is

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

States

$$Q = \{R, G, Y\}$$

Alphabet

$$\Sigma = \{T\}$$

Transition Function

Current State	Input (T)	Next State
→R	T	G
G	T	Y
Y	T	R

Initial State

$$q_0 = R$$

Final State

Since the traffic light continuously cycles, there is **no final (accepting) state**.

$$F = \emptyset$$

Transition Table

Current State	Input	Next State
→ Red	Timer	Green
Green	Timer	Yellow
Yellow	Timer	Red

Timing Constraints

Each signal remains active for a predetermined duration.

State	Typical Time
-------	--------------

Red	60 seconds
-----	------------

Green	45 seconds
-------	------------

Yellow	5 seconds
--------	-----------

The timer generates the input **T**, causing transitions between states.

Continuous Cycling

The automaton operates continuously.

- Red remains active for its timer duration.
- After the timer expires, it changes to Green.
- Green changes to Yellow after its timer.
- Yellow changes back to Red.
- This process repeats indefinitely, ensuring smooth traffic flow.

Extension for Pedestrian Crossing

A pedestrian request button (**P**) can be added.

When **P** is pressed:

- The current Green signal completes safely.
- The system changes to Yellow.
- Then Red is activated.
- Pedestrians receive a **Walk** signal.
- After the pedestrian timer expires, the normal traffic cycle resumes.

This prevents accidents and allows safe road crossing.

Extension for Emergency Vehicle Override

Introduce an emergency signal (**E**).

When an emergency vehicle is detected:

- The controller immediately switches the required road to **Green**.
- All conflicting directions remain **Red**.
- After the emergency vehicle passes, the system resumes the normal cycle from the appropriate state.

This ensures quick and safe passage for emergency vehicles such as ambulances and fire trucks.

Applications

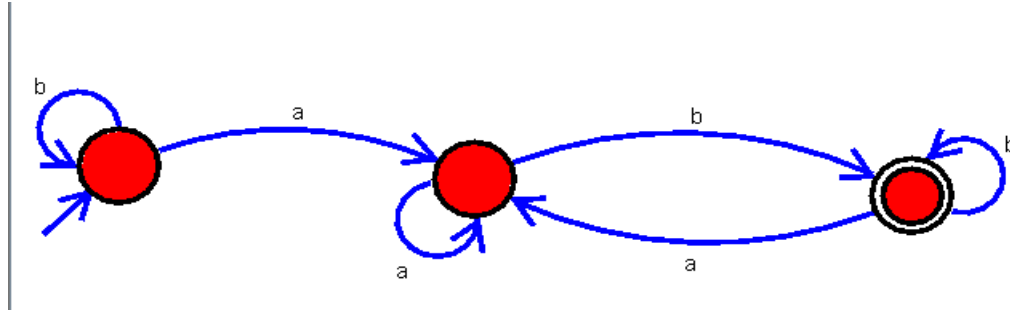
- Road intersections
- Smart traffic management systems
- Adaptive traffic controllers
- Intelligent transportation systems (ITS)
- Automated signal control

Conclusion

The designed **Finite Automaton** accurately models a traffic light control system using the states **Red**, **Green**, and **Yellow**. The transitions are controlled by a timer, ensuring the sequence **Red** → **Green** → **Yellow** → **Red** repeats continuously. The model can be

extended to support **pedestrian crossing signals** and **emergency vehicle overrides**, making the system safer, more efficient, and suitable for modern intelligent traffic control.

3. Design DFA using simulator to accept the string the end with ab over set {a,b} W= aaabab



Language Description

Design a Deterministic Finite Automaton (DFA) over the alphabet

$$\Sigma = \{a, b\}$$

that accepts all strings whose last two characters are ab.

Examples:

- ab ✓
- aaab ✓
- bab ✓
- aaabab ✓
- abaa ✗
- abb ✗

DFA Design

States

- q_0 – Start state
- q_1 – Last symbol is a
- q_2 – Last two symbols are ab (Accepting)

Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

States

$$Q = \{q_0, q_1, q_2\}$$

Alphabet

$$\Sigma = \{a, b\}$$

Start State

$$q_0$$

Final State

$$F = \{q_2\}$$

Transition Table

Current State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Simulation of the Input String

Given string

$$W = \text{aaabab}$$

Step	Input	Current State
Start	—	q_0
1	a	q_1
2	a	q_1
3	a	q_1
4	b	q_2
5	a	q_1
6	b	q_2

Final state reached:

$$q_2$$

Since q_2 is an accepting state,

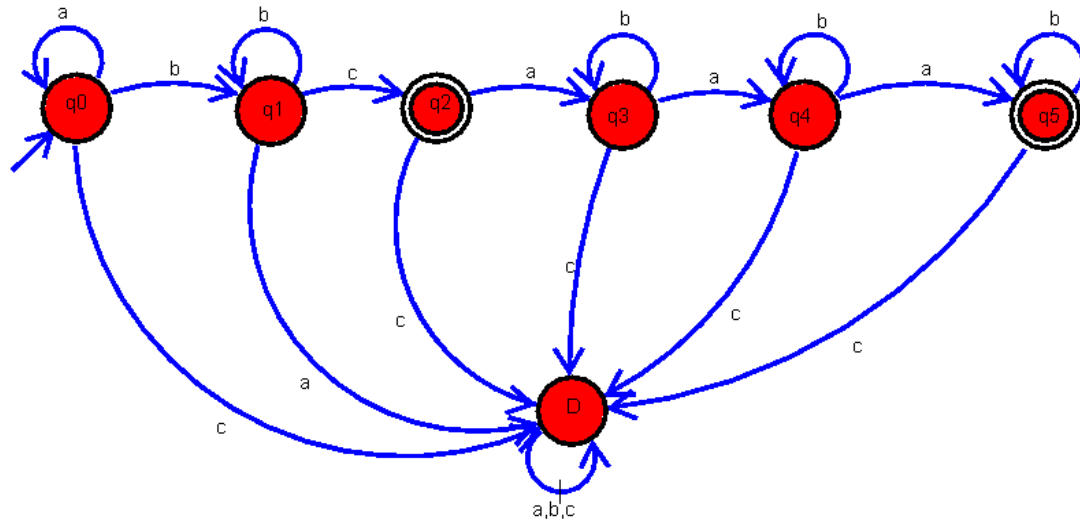
The string aaabab is Accepted.

Conclusion

The DFA correctly accepts all strings ending with ab. The input string aaabab ends with ab, so after simulation, the automaton reaches the accepting state q_2 . Therefore, the string is accepted.

- Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc”

should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.



Conceptual Understanding

A **Deterministic Finite Automaton (DFA)** is a finite-state machine in which every state has exactly one transition for each input symbol.

The DFA should accept only:

- c
- bc
- bcaaa

Any deviation from these strings must lead to a **Dead State (D)**.

DFA Design

States

- q_0 = Start State
- q_1 = After reading b
- q_2 = Accepted c or bc
- q_3 = After reading bca
- q_4 = After reading bcaa
- q_5 = Accepted bcaaa
- D = Dead State

Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

States

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, D\}$$

Alphabet

$$\Sigma = \{a, b, c\}$$

Start State

q_0

Final States

$F = \{q_2, q_5\}$

Transition Table

Current State	a	b	c
$\rightarrow q_0$	D	q_1	q_2^*
q_1	D	D	q_2^*
$*q_2$	q_3	D	D
q_3	q_4	D	D
q_4	q_5^*	D	D
$*q_5$	D	D	D
D	D	D	D

(* = Accepting State)

Conclusion

The DFA correctly accepts only the strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. Each valid string follows a unique sequence of transitions from the start state to an accepting state, while any invalid input immediately transitions to the **Dead State**, ensuring that no other strings are accepted.

5. Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

